C# Fundamentals – Data Types, Operators, Loops, Classes, Arrays, OOP, and File I/O

C# (pronounced "C-sharp") is one of the most widely used programming languages in the world. It powers Windows applications, web applications built on ASP.NET, mobile apps through Xamarin, game development through Unity, and enterprise software used by banks, hospitals, and governments. Unlike JavaScript, which runs inside the browser, C# runs on the .NET platform — a powerful framework developed by Microsoft for building robust, scalable, and secure applications. For students learning .NET, C# is the core language: it teaches structured programming, object-oriented design, and how to build real applications that store, process, and save data. Understanding C# fundamentals is essential for anyone who wants to pursue a career in software development.

This reading material introduces the fundamentals of C#. It discusses the history, features, and program structure of C#. It explains how to use C# data types, variables, strings, and constants. It covers C# operators and decision-making, then iteration statements. It explains classes, methods, and structure, followed by arrays. It then applies the object-oriented programming approach — including interfaces, encapsulation, polymorphism, and inheritance. Finally, it explains file input and output for creating, deleting, and reading data in web applications.

Part 1: C# Fundamentals – History, Features, and Program Structure

C# did not emerge in isolation. It was developed by Microsoft in the early 2000s as part of the .NET initiative — a response to the growing need for a modern, safe, and versatile programming language that could compete with Java while integrating tightly with the Windows platform. From its first release to its current versions, C# has evolved into a multi-paradigm language used across desktop, web, mobile, cloud, and game development. Understanding where C# came from and how it is structured helps students appreciate why it is designed the way it is.

History of C#

Year	Milestone
2000	Microsoft begins developing C# under the leadership of Anders Hejlsberg
2002	C# 1.0 released with .NET Framework 1.0
2005	C# 2.0 introduces generics, nullable types, and iterators
2007	C# 3.0 introduces LINQ, lambda expressions, and `var`
2010	C# 4.0 introduces dynamic typing and optional parameters
2012	C# 5.0 introduces async/await for asynchronous programming
2015	C# 6.0 introduces string interpolation and null-conditional operators
2017	C# 7.x introduces tuples, pattern matching, and local functions
2019	C# 8.0 introduces nullable reference types and async streams
2020	C# 9.0 introduces records and top-level statements
2021	C# 10 released with global usings and file-scoped namespaces
2022	C# 11 released with raw string literals and list patterns
2023	C# 12 released with primary constructors and collection expressions

Key figure: Anders Hejlsberg, a Danish software engineer, is the chief architect of C#. He also created Turbo Pascal and was involved in the development of TypeScript.

Features of C#

C# is designed to be modern, safe, and productive. Its features reflect these goals.

Feature	Description
Object-oriented	Supports classes, objects, inheritance, polymorphism, and encapsulation
Type-safe	Prevents many common programming errors by enforcing type rules
Managed memory	Garbage collector automatically frees unused memory
Cross-platform	Runs on Windows, Linux, and macOS through .NET Core / .NET 5+
Rich standard library	Provides built-in classes for common tasks
LINQ support	Allows querying collections and databases using a unified syntax
Asynchronous programming	`async`/`await` make it easy to write non-blocking code
Exception handling	`try`/`catch`/`finally` for robust error management
Interoperability	Can interact with other languages on the .NET platform
Open source	Modern .NET is open source and community-driven

Structure of a C# Program

Every C# program follows a consistent structure. Understanding this structure is the first step toward writing working code.

Basic Structure:

```csharp
using System;

namespace MyApplication
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, World!");
        }
    }
}
```

Explanation of Each Part:

Part	Purpose
`using System;`	Imports the `System` namespace, giving access to classes like `Console`
`namespace MyApplication`	Groups related classes under a named scope
`class Program`	Declares a class named `Program`
`static void Main(string[] args)`	The **entry point** — where execution begins
`Console.WriteLine(...)`	Prints text to the console

C# Syntax Rules

Rule	Explanation
Case-sensitive	`Name` and `name` are different identifiers
Semicolons	Every statement ends with `;`
Curly braces	`{ }` define code blocks
Comments	`//` for single-line; `/* ... */` for multi-line
Naming conventions	Classes use `PascalCase`; variables use `camelCase`
File extension	C# source files use `.cs`

A Simple Complete Program

```csharp
using System;

namespace GreetingApp
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.Write("Enter your name: ");
            string name = Console.ReadLine();
            Console.WriteLine("Hello, " + name + "!");
        }
    }
}
```

When run, the program asks for the user's name and prints a greeting.

Part 2: C# Data Types, Variables, Strings, and Constants

Every value stored in a C# program has a **data type**. The data type determines what kind of value the variable can hold, how much memory it uses, and what operations can be performed on it. C# is a **strongly typed** language — a variable's type must be declared before it is used. This prevents many common programming errors. Understanding data types, variables, strings, and constants is essential for writing correct C# code.

C# Data Types

C# data types are divided into **value types** (stored directly in memory) and **reference types** (stored as references to memory locations).

Common Value Types:

Type	Size	Range / Description	Example
`int`	4 bytes	-2,147,483,648 to 2,147,483,647	`int age = 17;`
`long`	8 bytes	Larger integers	`long population = 8000000000;`
`float`	4 bytes	Decimal with ~7 digits precision	`float price = 19.99f;`
`double`	8 bytes	Decimal with ~15 digits precision	`double pi = 3.14159;`
`decimal`	16 bytes	High-precision decimal for money	`decimal salary = 25000.50m;`
`bool`	1 byte	`true` or `false`	`bool isActive = true;`
`char`	2 bytes	A single Unicode character	`char grade = 'A';`
`byte`	1 byte	0 to 255	`byte level = 200;`

Common Reference Types:

Type	Description	Example
`string`	A sequence of characters	`string name = "Juan";`
`object`	Base type of all types	`object obj = 42;`
Arrays	Collection of elements	`int[] scores = {90, 85, 88};`
Classes	User-defined blueprints	`Student s = new Student();`

Variables

A **variable** is a named storage location in memory. In C#, variables must be declared with a type before use.

Syntax:

```csharp
type variableName = value;
```

Example:

```csharp
int age = 17;
string name = "Maria";
double grade = 92.5;
bool isPassed = true;
```

Variable Declaration Without Initialization:

```csharp
int score;
score = 85;
```

Naming Rules for Variables:

Rule	Example
Must start with a letter or underscore	`_count` (valid); `1count` (invalid)
Cannot contain spaces or special characters	`student_name` (valid); `student-name` (invalid)
Cannot be a C# keyword	`int` (invalid); `value` (valid)
Case-sensitive	`Name` and `name` are different
Use camelCase	`firstName`, `totalScore`

Constants

A **constant** is a variable whose value cannot be changed after initialization. Constants are declared using the `const` keyword.

Syntax:

```csharp
const type NAME = value;
```

Example:

```csharp
const double PI = 3.14159;
const int MAX_STUDENTS = 50;
```

Rules for Constants:

Rule	Explanation
Must be initialized at declaration	`const int X;` is invalid
Cannot be modified after declaration	`PI = 3.14;` causes a compile error

Rule	Explanation
Common convention	Constants use `UPPER_SNAKE_CASE`
Only compile-time constants	Value must be known at compile time

Strings

A **string** is a sequence of characters. Strings in C# are immutable — once created, they cannot be changed.

Creating Strings:

```csharp
string greeting = "Hello, World!";
string name = "Juan";
string empty = "";
```

Common String Operations:

Operation	Example	Result
Concatenation	`"Hello, " + name`	`"Hello, Juan"`
Length	`name.Length`	`4`
Uppercase	`name.ToUpper()`	`"JUAN"`
Lowercase	`name.ToLower()`	`"juan"`
Substring	`name.Substring(0, 2)`	`"Ju"`
Index access	`name[0]`	`'J'`
Replace	`name.Replace("J", "M")`	`"Muan"`
Contains	`name.Contains("ua")`	`true`

Operation	Example	Result
Trim	`" Juan ".Trim()`	`"Juan"`
Split	`"a,b,c".Split(',')`	`["a", "b", "c"]`

String Interpolation (Recommended):

```csharp
string name = "Maria";
int age = 17;
string message = $"Hello, my name is {name} and I am {age} years old.";
Console.WriteLine(message);
```

String interpolation uses `$""` and `{}` to embed expressions directly.

Verbatim Strings:

```csharp
string path = @"C:\Users\Juan\Documents\file.txt";
```

The `@` prefix prevents escape sequence processing.

Type Conversion

C# allows conversion between data types, either **implicitly** (safe conversions) or **explicitly** (with possible data loss).

Implicit Conversion (automatic):

```csharp
int num = 100;
double d = num;   // int to double — safe
```

Explicit Conversion (casting):

```csharp
double d = 9.78;
int i = (int)d;   // 9 — decimal part lost
```

Conversion Methods:

```csharp
string s = "123";
int num = int.Parse(s);          // 123
int num2;
int.TryParse(s, out num2);       // safer – returns bool

double d = Convert.ToDouble("3.14");
```

Part 3: C# Operators and Decision Making

C# provides a rich set of operators for performing calculations, comparisons, and logical evaluations. Combined with decision-making statements (`if`, `else`, `switch`), these operators allow programs to respond intelligently to data and user input. This section explains the main categories of C# operators and how they are used in decision-making.

Categories of C# Operators

Category	Purpose	Examples
Arithmetic	Mathematical operations	`+`, `-`, `*`, `/`, `%`
Assignment	Assign and update values	`=`, `+=`, `-=`, `*=`, `/=`
Comparison	Compare values	`==`, `!=`, `>`, `<`, `>=`, `<=`
Logical	Combine boolean conditions	`&&`, `||`, `!`
Increment / Decrement	Increase or decrease by 1	`++`, `--`
Bitwise	Operate on bits	`&`, `|`, `^`, `<<`, `>>`
Ternary	Short form of if-else	`condition ? a : b`

Arithmetic Operators

Operator	Meaning	Example	Result
`+`	Addition	`5 + 3`	`8`
`-`	Subtraction	`10 - 4`	`6`
`*`	Multiplication	`6 * 7`	`42`
`/`	Division	`20 / 4`	`5`
`%`	Modulus	`10 % 3`	`1`

Important note on division: In C#, dividing two integers produces an integer result. To get a decimal result, at least one operand must be a `double` or `decimal`.

```csharp
int a = 10;
int b = 3;
Console.WriteLine(a / b);        // 3 (integer division)
Console.WriteLine((double)a / b); // 3.333... (floating division)
```

Comparison Operators

Operator	Meaning	Example	Result
`==`	Equal	`5 == 5`	`true`
`!=`	Not equal	`5 != 3`	`true`
`>`	Greater than	`10 > 5`	`true`
`<`	Less than	`10 < 5`	`false`
`>=`	Greater than or equal	`10 >= 10`	`true`

Operator	Meaning	Example	Result
`<=`	Less than or equal	`5 <= 3`	`false`

Logical Operators

Operator	Meaning	Example	Result
`&&`	AND — both true	`true && false`	`false`
`||`	OR — at least one true	`true || false`	`true`
`!`	NOT — reverses	`!true`	`false`

Decision Making with `if`, `else if`, `else`

```csharp
int score = 85;

if (score >= 90)
{
    Console.WriteLine("Outstanding");
}
else if (score >= 85)
{
    Console.WriteLine("Very Satisfactory");
}
else if (score >= 75)
{
    Console.WriteLine("Satisfactory");
}
else
{
    Console.WriteLine("Failed");
}
```

The `switch` Statement

The `switch` statement compares a single expression to multiple cases.

```csharp
int day = 3;

switch (day)
{
    case 1:
        Console.WriteLine("Monday");
        break;
    case 2:
        Console.WriteLine("Tuesday");
        break;
    case 3:
        Console.WriteLine("Wednesday");
        break;
    default:
        Console.WriteLine("Other day");
        break;
}
```

Rules for `switch`:

Rule	Explanation
`case` labels must be constant	Cannot use variables or expressions
Each case ends with `break`	Prevents fall-through
`default` handles unmatched cases	Optional but recommended
C# 7+ allows pattern matching	`case int n when n > 0:`

The Ternary Operator

The ternary operator is a compact way to write a simple `if-else`.

```csharp
int age = 18;
string status = age >= 18 ? "Adult" : "Minor";
Console.WriteLine(status);   // Adult
```

Comparison: `if` vs. `switch`

Situation	Recommended
Testing ranges (e.g., `score >= 75`)	`if...else`
Testing boolean conditions	`if...else`
Testing many exact values of one variable	`switch`
Complex conditions with `&&`, `||`	`if...else`

Part 4: C# Loops (Iteration)

Loops allow a program to repeat a block of code without duplicating it. C# provides four main loops: `while`, `do...while`, `for`, and `foreach`. Each serves a specific purpose depending on whether the iteration count is known and whether the loop must run at least once.

The `while` Loop

The `while` loop checks its condition **before** each iteration.

```
csharp
int i = 1;
while (i <= 5)
{
    Console.WriteLine(i);
    i++;
}
```

Use case: When the number of iterations is unknown in advance.

The `do...while` Loop

The `do...while` loop runs the body at least once, then checks the condition.

```
csharp
int number;
do
{
    Console.Write("Enter a number greater than 10: ");
    number = int.Parse(Console.ReadLine());
} while (number <= 10);
```

Use case: When the body must run at least once (input validation).

The `for` Loop

The `for` loop is used when the number of iterations is known.

```
csharp
for (int i = 1; i <= 5; i++)
{
    Console.WriteLine(i);
}
```

Structure of a `for` loop:

Part	Purpose	Example
Initialization	Runs once at start	`int i = 1`
Condition	Checked before each iteration	`i <= 5`
Increment	Runs after each iteration	`i++`

The `foreach` Loop

The `foreach` loop iterates over each element in a collection (arrays, lists).

```csharp
string[] fruits = { "apple", "banana", "mango" };

foreach (string fruit in fruits)
{
    Console.WriteLine(fruit);
}
```

Use case: Iterating through arrays and collections without managing an index.

The `break` and `continue` Keywords

Keyword	Effect
`break`	Exits the loop entirely
`continue`	Skips the rest of the current iteration

```csharp
for (int i = 1; i <= 10; i++)
{
    if (i == 5) break;
    Console.WriteLine(i);    // prints 1–4
```

```
}

for (int i = 1; i <= 5; i++)
{
    if (i == 3) continue;
    Console.WriteLine(i);    // prints 1, 2, 4, 5
}
```

Nested Loops

A loop can contain another loop.

```
csharp
for (int i = 1; i <= 3; i++)
{
    for (int j = 1; j <= 3; j++)
    {
        Console.Write($"{i},{j}  ");
    }
    Console.WriteLine();
}
```

Output:

```
text
1,1  1,2  1,3
2,1  2,2  2,3
3,1  3,2  3,3
```

Comparison of Loops

Loop	Condition Check	Runs at Least Once?	Best For
`while`	Before	No	Unknown iteration count
`do...while`	After	Yes	Must run at least once
`for`	Before	No	Known iteration count

Loop	Condition Check	Runs at Least Once?	Best For
`foreach`	Before	No	Iterating collections

Infinite Loops

An infinite loop never terminates because its condition never becomes false. Always ensure the condition eventually fails.

```csharp
// Infinite loop — avoid!
int i = 1;
while (i <= 5)
{
    Console.WriteLine(i);
    // missing i++
}
```

Part 5: C# Classes, Methods, and Structure

C# is an **object-oriented** language. Its fundamental building block is the **class** — a blueprint that defines both data (fields and properties) and behavior (methods). Understanding classes, methods, and structure is essential for organizing programs into reusable, maintainable pieces.

What Is a Class?

A **class** is a template or blueprint for creating objects. It defines:

- **Fields** — variables that store data
- **Properties** — controlled access to fields
- **Methods** — actions the object can perform
- **Constructors** — special methods that initialize objects

Class Syntax:

```csharp
public class Student
{
    // Fields
    private string name;
    private int age;

    // Constructor
    public Student(string name, int age)
    {
        this.name = name;
        this.age = age;
    }

    // Method
    public void Introduce()
    {
        Console.WriteLine($"Hi, I'm {name} and I'm {age} years old.");
    }
}
```

Fields vs. Properties

Concept	Description	Example
Field	Direct data member of a class	`private string name;`
Property	Controlled access through `get`/`set`	`public string Name { get; set; }`

Property Example:

```csharp
public class Student
{
    private int age;

    public int Age
    {
        get { return age; }
        set
        {
            if (value >= 0)
                age = value;
        }
    }
}
```

Auto-implemented properties simplify the syntax:

```csharp
public string Name { get; set; }
```

Methods

A **method** is a block of code that performs a specific task. Methods have:

- A return type (or `void` if no value is returned)
- A name
- Parameters (optional)
- A body

Method Syntax:

```csharp
accessModifier returnType MethodName(parameters)
{
    // body
}
```

Example:

```csharp
public int Add(int a, int b)
{
    return a + b;
}
```

Creating and Using Objects

To use a class, an **object** must be created using the `new` keyword.

```csharp
Student student1 = new Student("Juan", 17);
student1.Introduce();   // Hi, I'm Juan and I'm 17 years old.
```

Constructors

A **constructor** is a special method that runs when an object is created.

```csharp
public class Student
{
    public string Name;
    public int Age;

    public Student(string name, int age)
    {
        Name = name;
        Age = age;
    }
}
```

Types of Constructors:

Type	Description
Default constructor	No parameters; runs when object is created without values
Parameterized constructor	Takes parameters to initialize fields
Static constructor	Runs once when the class is first used
Copy constructor	Creates a new object from an existing object

Structs

A **struct** is similar to a class but is a **value type** (stored directly in memory) rather than a reference type. Structs are used for small, simple data structures.

```
csharp
public struct Point
{
    public int X;
    public int Y;

    public Point(int x, int y)
    {
        X = x;
        Y = y;
    }
}
```

Class vs. Struct:

Aspect	Class	Struct
Type	Reference type	Value type
Memory	Heap	Context-dependent

Aspect	Class	Struct
Inheritance	Supported	Not supported
Use case	Complex objects	Small, simple data

Access Modifiers

Modifier	Description
`public`	Accessible from anywhere
`private`	Accessible only within the class
`protected`	Accessible within the class and derived classes
`internal`	Accessible within the same assembly
`protected internal`	Accessible within the assembly or derived classes

Static Members

Members marked `static` belong to the class itself, not to any object.

```csharp
public class MathHelper
{
    public static int Square(int x)
    {
        return x * x;
    }
}

Console.WriteLine(MathHelper.Square(5));    // 25
```

Part 6: C# Arrays

An **array** is a fixed-size collection of elements of the same type. Arrays store multiple values under a single name and access each element by its **index**, starting at 0. C# arrays are **strongly typed** — an array declared as `int[]` can only hold integers. Arrays are essential for organizing data such as lists of scores, names, or prices.

Declaring Arrays

Syntax:

```
csharp
type[] arrayName;
```

Example:

```
csharp
int[] scores;           // declaration only
int[] scores2 = new int[5];   // array of 5 integers, all default to 0
int[] scores3 = { 90, 85, 88, 92, 78 };  // initialized with values
int[] scores4 = new int[] { 90, 85, 88 };
```

Accessing Array Elements

```
csharp
int[] scores = { 90, 85, 88, 92, 78 };

Console.WriteLine(scores[0]);   // 90
Console.WriteLine(scores[4]);   // 78
```

Array Length

```
csharp
Console.WriteLine(scores.Length);   // 5
```

Modifying Array Elements

```csharp
scores[1] = 95;
Console.WriteLine(scores[1]);   // 95
```

Iterating Over Arrays

```csharp
int[] scores = { 90, 85, 88, 92, 78 };

// Using for loop
for (int i = 0; i < scores.Length; i++)
{
    Console.WriteLine(scores[i]);
}

// Using foreach loop
foreach (int score in scores)
{
    Console.WriteLine(score);
}
```

Common Array Operations

Operation	Description	Example
`Array.Sort()`	Sorts elements in ascending order	`Array.Sort(scores);`
`Array.Reverse()`	Reverses the order	`Array.Reverse(scores);`
`Array.IndexOf()`	Finds the first index of a value	`Array.IndexOf(scores, 88)`
`Array.Copy()`	Copies elements to another array	`Array.Copy(a, b, 5);`
`Array.Clear()`	Resets elements to default	`Array.Clear(scores, 0, 5);`

Example: Sorting and Reversing

```csharp
int[] scores = { 90, 85, 88, 92, 78 };

Array.Sort(scores);
foreach (int s in scores) Console.WriteLine(s);
// 78, 85, 88, 90, 92

Array.Reverse(scores);
foreach (int s in scores) Console.WriteLine(s);
// 92, 90, 88, 85, 78
```

Multi-Dimensional Arrays

C# supports 2D and higher-dimensional arrays.

2D Array:

```csharp
int[,] matrix = new int[3, 3];

matrix[0, 0] = 1;
matrix[0, 1] = 2;

int[,] predefined = {
    { 1, 2, 3 },
    { 4, 5, 6 },
    { 7, 8, 9 }
};

Console.WriteLine(predefined[1, 2]);   // 6
```

Iterating a 2D Array:

```csharp
for (int i = 0; i < 3; i++)
{
    for (int j = 0; j < 3; j++)
    {
```

```
        Console.Write(predefined[i, j] + " ");
    }
    Console.WriteLine();
}
```

Jagged Arrays

A **jagged array** is an array of arrays. Each inner array can have a different length.

```csharp
int[][] jagged = new int[3][];
jagged[0] = new int[] { 1, 2 };
jagged[1] = new int[] { 3, 4, 5 };
jagged[2] = new int[] { 6 };

foreach (int[] arr in jagged)
{
    foreach (int num in arr) Console.Write(num + " ");
    Console.WriteLine();
}
```

Arrays vs. Lists

Aspect	Array	List
Size	Fixed	Dynamic
Type	`int[]`	`List<int>`
Methods	Limited	Rich (Add, Remove, Insert, etc.)
Namespace	`System`	`System.Collections.Generic`
Best for	Known-size collections	Growing/shrinking collections

---

Part 7: C# Object-Oriented Programming (OOP)

Object-oriented programming (OOP) is a design approach in which programs are organized around **objects** rather than functions and logic. C# is built on OOP principles. The four pillars of OOP are **encapsulation**, **inheritance**, **polymorphism**, and **abstraction** (often implemented through **interfaces**). Mastering OOP is essential for writing scalable, maintainable applications.

The Four Pillars of OOP

Pillar	Description
Encapsulation	Bundling data and methods that operate on the data into a single unit (class); restricting direct access to some components
Inheritance	Creating new classes based on existing ones, reusing and extending behavior
Polymorphism	The ability of objects to take on multiple forms — same method name, different behavior
Abstraction	Hiding implementation details and exposing only essential features

Encapsulation

Encapsulation protects an object's data from outside interference by hiding fields behind properties or methods.

```
csharp
public class BankAccount
{
    private decimal balance;

    public void Deposit(decimal amount)
```

```
    {
        if (amount > 0)
            balance += amount;
    }

    public decimal GetBalance()
    {
        return balance;
    }
}
```

Benefits of encapsulation:

Benefit	Explanation
Data protection	Fields cannot be modified directly
Controlled access	Validation can be applied in setters
Maintainability	Internal implementation can change without breaking external code

Inheritance

Inheritance allows a class (the **child** or **derived** class) to inherit fields and methods from another class (the **parent** or **base** class).

```
csharp
public class Animal
{
    public string Name { get; set; }

    public virtual void Speak()
    {
        Console.WriteLine("Some animal sound");
    }
}

public class Dog : Animal
```

```
{
    public override void Speak()
    {
        Console.WriteLine("Woof!");
    }
}

public class Cat : Animal
{
    public override void Speak()
    {
        Console.WriteLine("Meow!");
    }
}
```

Benefits of inheritance:

Benefit	Explanation
Code reuse	Common behavior lives in the parent class
Extensibility	New behavior can be added in child classes
Organization	Related classes form a clear hierarchy

Polymorphism

Polymorphism allows objects of different classes to be treated as objects of a common base class, with each class providing its own version of a method.

```
csharp
Animal[] animals = {
    new Dog { Name = "Rex" },
    new Cat { Name = "Whiskers" }
};

foreach (Animal a in animals)
{
```

```
    a.Speak();   // Rex: Woof!  Whiskers: Meow!
}
```

Types of Polymorphism:

Type	Description	Example
Compile-time (overloading)	Same method name, different parameters	`Add(int, int)` vs. `Add(double, double)`
Runtime (overriding)	Child class overrides parent method	`Dog.Speak()` overrides `Animal.Speak()`

Method Overloading Example:

```
csharp
public int Add(int a, int b) => a + b;
public double Add(double a, double b) => a + b;
public int Add(int a, int b, int c) => a + b + c;
```

Abstraction and Interfaces

Abstraction hides implementation details behind a simpler interface. In C#, abstraction is achieved through **abstract classes** and **interfaces**.

Abstract Class:

```
csharp
public abstract class Shape
{
    public abstract double Area();
}

public class Circle : Shape
{
    public double Radius { get; set; }

    public override double Area()
    {
```

```
        return Math.PI * Radius * Radius;
    }
}
```

Interface:

An **interface** defines a contract — a set of methods and properties that any class implementing the interface must provide.

```
csharp
public interface IVehicle
{
    void Start();
    void Stop();
}

public class Car : IVehicle
{
    public void Start()
    {
        Console.WriteLine("Car started.");
    }

    public void Stop()
    {
        Console.WriteLine("Car stopped.");
    }
}
```

Interface vs. Abstract Class:

Aspect	Interface	Abstract Class
Multiple inheritance	Allowed	Not allowed
Implementation	Cannot contain implementation (until C# 8)	Can contain concrete method
Fields	Not allowed	Allowed

Aspect	Interface	Abstract Class
Purpose	Defines a contract	Provides partial implementati
Use case	Unrelated classes sharing behavior	Related classes sharing code

Summary Table: OOP in C#

Concept	Keyword	Purpose
Class	`class`	Blueprint for objects
Object	`new`	Instance of a class
Encapsulation	`private`, `public`, properties	Protect and control data access
Inheritance	`:` (colon)	Reuse parent class behavior
Overriding	`virtual`, `override`	Replace parent method
Overloading	Same name, different parameters	Multiple method versions
Abstract class	`abstract`	Partial base class
Interface	`interface`	Contract of methods

Part 8: C# File Input/Output

Most real applications need to **save** data permanently and **read** it back later. C# provides the `System.IO` namespace for reading from and writing to files. File input/output (I/O) allows applications to create, delete, read, and write files — an essential capability for storing user data, logs, configurations, and reports.

The `System.IO` Namespace

The `System.IO` namespace contains classes for working with files and directories.

Class	Purpose
`File`	Static methods for creating, deleting, and checking files
`FileInfo`	Instance methods for file information
`Directory`	Static methods for directories
`StreamReader`	Reads text from a file
`StreamWriter`	Writes text to a file
`FileStream`	Low-level file stream access
`Path`	Utilities for file paths

To use these classes, add:

```csharp
using System.IO;
```

Writing to a File

The `File.WriteAllText()` method writes a string to a file. If the file exists, it is overwritten; if not, it is created.

```csharp
using System.IO;

string path = @"C:\temp\notes.txt";
File.WriteAllText(path, "Hello, this is a test.");
```

Writing multiple lines:

```csharp
string[] lines = { "Line 1", "Line 2", "Line 3" };
File.WriteAllLines(path, lines);
```

Reading from a File

The `File.ReadAllText()` method reads the entire contents of a file into a string.

```csharp
string content = File.ReadAllText(path);
Console.WriteLine(content);
```

Reading all lines:

```csharp
string[] lines = File.ReadAllLines(path);
foreach (string line in lines)
{
    Console.WriteLine(line);
}
```

Appending to a File

`File.AppendAllText()` adds text without overwriting.

```csharp
File.AppendAllText(path, "\nNew line appended.");
```

Checking if a File Exists

```csharp
if (File.Exists(path))
{
    Console.WriteLine("File exists.");
}
else
{
    Console.WriteLine("File not found.");
}
```

Deleting a File

```csharp
if (File.Exists(path))
{
    File.Delete(path);
    Console.WriteLine("File deleted.");
}
```

Using `StreamWriter` and `StreamReader`

For more control, use `StreamWriter` and `StreamReader` inside a `using` block so that resources are automatically disposed of.

Writing with `StreamWriter`:

```csharp
using (StreamWriter writer = new StreamWriter(path))
{
    writer.WriteLine("Line 1");
    writer.WriteLine("Line 2");
    writer.WriteLine("Line 3");
}
```

Reading with `StreamReader`:

```csharp
using (StreamReader reader = new StreamReader(path))
{
    string line;
    while ((line = reader.ReadLine()) != null)
    {
        Console.WriteLine(line);
    }
}
```

The `using` Statement

The `using` statement ensures that file handles are released automatically, even if an exception occurs. This prevents file locks and memory leaks.

```csharp
using (StreamReader reader = new StreamReader(path))
{
    // read from reader
}
// reader is automatically disposed here
```

Exception Handling with File I/O

File operations can fail for many reasons — the file does not exist, permission is denied, or the disk is full. Always wrap file operations in `try`/`catch` blocks.

```csharp
try
{
    string content = File.ReadAllText(path);
    Console.WriteLine(content);
}
catch (FileNotFoundException)
{
    Console.WriteLine("The file was not found.");
}
catch (IOException ex)
{
    Console.WriteLine("An I/O error occurred: " + ex.Message);
}
```

File I/O in a Web Application

In ASP.NET web applications, file I/O is used for:

Use Case	Example
Logging	Writing errors and events to a log file
Configuration	Reading settings from a config file
Data export	Saving user data as CSV or text files
File uploads	Saving uploaded files to a folder
Reports	Generating text reports for download

Example — Logging to a File:

```csharp
public void LogError(string message)
{
    string logPath = @"C:\logs\app.log";
    string entry = $"{DateTime.Now}: {message}";
    File.AppendAllText(logPath, entry + Environment.NewLine);
}
```

Security considerations:

Concern	Recommendation
Path traversal	Validate user-supplied file paths
Permissions	Restrict write access to specific folders
Sensitive data	Do not log passwords or personal information
File size	Enforce size limits on uploads

Summary of Common File Methods

Task	Method
Write text	`File.WriteAllText(path, text)`
Read text	`File.ReadAllText(path)`
Write lines	`File.WriteAllLines(path, lines)`
Read lines	`File.ReadAllLines(path)`
Append text	`File.AppendAllText(path, text)`
Check existence	`File.Exists(path)`

Task	Method
Delete file	`File.Delete(path)`
Copy file	`File.Copy(src, dest)`
Move file	`File.Move(src, dest)`

Key Terms

Term	Definition
C#	A modern, object-oriented programming language developed by Microsoft for the .NET platform
.NET	A software framework by Microsoft that supports building and running applications
Namespace	A container that groups related classes
Entry point	The `Main` method — where program execution begins
Value type	A data type stored directly in memory (e.g., `int`, `bool`)
Reference type	A data type stored as a reference to a memory location (e.g., `string`, classes)
Variable	A named storage location for a value
Constant	A variable whose value cannot change after initialization
String interpolation	Embedding expressions inside a string using `$""`
Ternary operator	A compact `if-else` using `condition ? a : b`
Loop	A structure that repeats a block of code
Class	A blueprint that defines data and behavior
Object	An instance of a class

Term	Definition
Constructor	A special method that initializes an object
Encapsulation	Bundling data and methods and restricting direct access
Inheritance	Creating a new class based on an existing one
Polymorphism	The ability of objects to take multiple forms
Abstraction	Hiding implementation details behind a simpler interface
Interface	A contract that defines methods a class must implement
Array	A fixed-size collection of elements of the same type
File I/O	Reading from and writing to files

Review Questions

1. Explain the history of C# and identify one key figure in its development. State two features that make C# a modern programming language.

2. Differentiate value types from reference types in C#. Provide two examples of each.

3. Explain the difference between a variable and a constant in C#. Provide one example of each.

4. Differentiate `==` and `=` in C#. Explain when each should be used.

5. Explain the difference between `while` and `do...while` loops. Provide one situation where `do...while` is more appropriate.

6. Differentiate a class from a struct in C#. State one situation where each is more appropriate.

7. Explain the four pillars of object-oriented programming. Provide one sentence for each.

8. Differentiate an interface from an abstract class. Provide one advantage of each.

9. Study the code below and answer the questions.

```csharp
using System;
using System.IO;

class Program
{
    static void Main()
    {
        string path = @"C:\temp\scores.txt";
        int[] scores = { 90, 85, 88 };

        using (StreamWriter writer = new StreamWriter(path))
        {
            foreach (int s in scores)
```

```
                writer.WriteLine(s);
        }

        if (File.Exists(path))
            Console.WriteLine("File saved.");
    }
}
```

- (a) What does the code do?
- (b) What does the `using` block ensure?
- (c) If the folder `C:\temp` does not exist, what will happen?

10. Application question: A school wants to create a web application that allows teachers to record students' grades into a text file and later read them back. Using the concepts from this lesson (classes, arrays, file I/O), describe how you would design the application. Include the class you would create, the methods it would contain, and how file I/O would be used to save and load data.

Answer Key

1. C# was developed by Microsoft in 2000 and released in 2002 with .NET Framework 1.0. Anders Hejlsberg is its chief architect. Two modern features are garbage collection (managed memory) and LINQ support (unified querying).

2. Value types store data directly in memory and include `int`, `bool`, `double`, and `char`. Reference types store a reference to a memory location and include `string`, arrays, and class objects.

3. A variable can be reassigned after declaration (e.g., `int age = 17; age = 18;`). A constant cannot be changed after initialization (e.g., `const double PI = 3.14159;`).

4. `=` is the assignment operator and assigns a value to a variable (`int x = 5;`). `==` is the comparison operator and returns `true` or `false` based on equality (`if (x == 5)`).

5. A `while` loop checks the condition before running the body, so the body may never run. A `do...while` loop runs the body at least once before checking. `do...while` is more appropriate for input validation, where the user must be asked at least once.

6. A class is a reference type and supports inheritance. A struct is a value type and does not support inheritance from other classes or structs. Use a class for complex objects with behavior; use a struct for small, simple data (e.g., a `Point` with X and Y).

7. Encapsulation bundles data and methods and restricts direct access. Inheritance allows a class to reuse and extend another class. Polymorphism lets objects of different classes respond to the same method call. Abstraction hides implementation details behind a simpler interface.

8. An interface defines a contract that a class agrees to implement. A class can implement multiple interfaces, while C# does not support inheritance from multiple classes. An abstract class can contain both abstract and concrete methods and allows shared implementation. An advantage of an interface is flexibility across unrelated classes; an advantage of an abstract class is code sharing.

9.

(a) The code writes the numbers 90, 85, and 88 to a text file, then checks if the file exists and prints "File saved."

(b) The `using` block ensures that the `StreamWriter` is properly closed and disposed of, releasing the file handle even if an exception occurs.

(c) A `DirectoryNotFoundException` will be thrown because the folder does not exist. The program should check and create the directory first with `Directory.CreateDirectory(...)`.

10.

- (a) Create a class `GradeBook` with fields or properties such as `List<Student>` or `Student[]` and a `string FilePath`.
- (b) Create a class `Student` with properties `Name` and `Grade`.
- (c) Add methods: `AddGrade(string name, int grade)`, `SaveToFile()`, `LoadFromFile()`, and `DisplayAll()`.
- (d) `SaveToFile()` uses `File.WriteAllLines` or `StreamWriter` to write each student's name and grade to a text file.
- (e) `LoadFromFile()` uses `File.ReadAllLines` or `StreamReader` to read data back into the array or list.
- (f) Use `try`/`catch` for error handling and the `using` statement to ensure proper disposal of file resources.